Manuel de programmation - pili-pos-android v1.0.0

Plateforme : Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google)

Ce manuel s'adresse au distributeur (nuagemagique.dev) ou à un intégrateur tiers autorisé. Il décrit la **configuration du catalogue** (catégories, articles, modificateurs, traductions, codes TVA) et la **gestion des utilisateurs**.

Attention : **Ces opérations ne sont PAS accessibles à l'opérateur du restaurant.** Le terminal n'a ni console, ni clavier de maintenance, ni compte de contrôle, ni fichier à éditer, ni accès à distance (`PLATEFORME_ANDROID_FR.md` §3 et §7). Le catalogue et la configuration initiale se préparent sur le poste du producteur et entrent dans la caisse par le **lot de mise en service signé** (`MANUEL_INSTALLATION_FR.md` §2 et §3.4) : `pos.json` et, facultativement, `seed.sql`. Ensuite, un Gérant gère les employés dans "Config" (§6) et, quand HubRise gère le catalogue, le menu par HubRise (§4.5). Sinon le Gérant crée et modifie la carte depuis "Config", panneau "Carte" (`EDITEUR_CARTE_FR.md`). Les réglages de l'écran sont au §15.

1. Accès en mode programmation

1.1 Poste du producteur et lot signé

Il n'y a pas d'accès en mode programmation sur la caisse. Le producteur écrit sur son poste `pos.json` (`MANUEL_INSTALLATION_FR.md` §4.2) et, si le catalogue ou les premiers comptes sont livrés d'avance, `seed.sql` (§3 à §6), génère `MANIFEST.json` (fichiers triés, taille et SHA-256), le signe avec sa clé de production (`minisign -S`) et importe le lot au banc (`MANUEL_INSTALLATION_FR.md` §3.4).

À l'import, l'application vérifie la signature et chaque empreinte, contrôle `pos.json`, puis applique `seed.sql` **une seule fois, à une base qui n'existe pas encore** (format du lot : `docs/android/APP.md` §3). Si la base existe déjà, la graine est ignorée. Le moteur la refuse aussi dès qu'un événement fiscal est enregistré ou que le menu est géré par une source externe, et la charge en une transaction : tout ou rien.

1.2 Outils disponibles

Sur le poste du producteur, pas sur la caisse : `sqlite3` (écrire et essayer `seed.sql` sur une base neuve de travail), `minisign` (signer le manifeste) et `scripts/android/make_dev_bundle.sh` (lot de **développement** avec les simulateurs, `--seed` pour la graine du jeu de test du SPF, jamais un lot de livraison). Les journaux de la caisse ne se lisent que par l'export vers le SPF (§12).

2. Gestion des langues

La base est livrée avec 5 langues actives : `fr`, `nl`, `de`, `en`, `zh`. Pour en désactiver une (sans la supprimer), l'ajouter à `seed.sql` :

```sql
UPDATE languages SET active = 0 WHERE code = 'nl';
```

L'écran propose le français, l'anglais et le chinois (panneau "Langue", §15) pour les noms des produits et des catégories ; la langue des mentions imprimées est la clé `language` de `pos.json` (`GARANTIE_MAJ_LANGUES_FR.md` §3).

3. Gestion des catégories de menu

Les instructions des §3 à §5 s'écrivent dans `seed.sql`. Un fichier de graine ne relit pas de résultat intermédiaire (pas de last_insert_rowid) : donner **des identifiants explicites**.

```sql
INSERT INTO categories (id, sort_order, color, active)
VALUES (5, 1, '#F2A9BD', 1);

-- Traductions du nom (au moins fr et zh)
INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
```

```
    ('category', 5, 'name', 'fr', 'Desserts'),
    ('category', 5, 'name', 'zh', '□ □ ');
```

`sort_order` et `active` (1 ou 0) se fixent dans l'`INSERT`. Sur une caisse en service, ni l'ordre ni l'état d'une catégorie ne se changent plus par SQL : seul HubRise le fait (§4.5).

Attention : Ne JAMAIS faire de `DELETE FROM categories` si la catégorie contient des `menu_items` référencés dans des `order_lines` - la FK casse.

4. Gestion des articles (`menu_items`)

4.1 Créer un article

```
INSERT INTO menu_items
    (id, category_id, price_cents, vat_code, vat_category,
     kitchen_printer, sort_order, active)
VALUES
    (42, 5, 850, 'B', 'food', 1, 0, 1);
-- 850 centimes = 8,50 € ; vat_code B = 12% (service salle)
-- kitchen_printer 1 = ligne reprise sur le bon de commande (bar ou cuisine)

INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('menu_item', 42, 'name', 'fr', 'Tarte maison'),
    ('menu_item', 42, 'name', 'zh', '□ □ □ □ ');
```

Deux colonnes facultatives complètent l'article :

- `vat_code_takeaway` : code TVA appliqué quand l'article est vendu à emporter. `NULL` (valeur par défaut) : le code `vat_code` ("Sur place") s'applique aussi à emporter (exemple : `'C'` pour 12 % sur place et 6 % à emporter).
- `open_price` : 1 pour un article à prix libre. Le prix est alors saisi à la caisse ("Prix libre") et `price_cents` ne sert pas.

4.2 Codes TVA (rappel)

`vat_code`	Taux	`vat_category`	Usage
`A`	21 %	`beverage`	Boissons, alcools
`B`	12 %	`food`	Cuisine consommée en salle
`C`	6 %	`food`	À emporter (cuisine) : à renseigner dans `vat_code_takeaway` d'un article dont `vat_code` est `B`, il s'applique aux commandes à emporter. Ce n'est pas une catégorie à part : le code `vat_code` reste celui de la vente sur place.
`D`	0 %	`zero_rated`	Produits exonérés
`X`	-	`out_of_scope`	Hors champ TVA

4.3 Modifier le prix d'un article

Sur une caisse en service, un prix ne se change pas par SQL : modifier l'article dans HubRise (§4.5). Les `order_lines` existantes conservent le prix au moment de leur ajout (champ dénormalisé `unit_price_cents`) - le changement n'affecte que les futures commandes. C'est le comportement attendu fiscalement.

4.4 Désactiver un article (rupture, retrait carte)

Dans HubRise (§4.5), ou `active = 0` dans la graine. Il disparaît de la grille de menu pour les nouvelles commandes mais reste consultable dans l'historique fiscal.

4.5 Menu géré par HubRise

C'est la **seule voie d'évolution du catalogue d'une caisse en service**. Le Gérant reprend le catalogue dans "Config" (panneau "HubRise", bouton "Reprendre le menu", `MANUEL_UTILISATEUR_FR.md` §12.7) ; le catalogue de la caisse est alors le miroir de celui de HubRise. Chaque synchronisation **écrase** les articles, catégories et options gérés, et l'écran refuse de modifier le menu ("Menu géré par HubRise : les produits se modifient dans HubRise."). Sans HubRise, le catalogue reste celui de la graine livrée au banc : le faire évoluer sur place suppose HubRise activé (`COMMANDES_EN_LIGNE_FR.md`).

5. Gestion des modificateurs

Les groupes d'options sont proposés à l'écran de commande (fenêtre "Options") quand HubRise est activé sur l'installation. Quand HubRise est désactivé, un article s'ajoute directement, sans fenêtre d'options. Dans la graine, avec des identifiants explicites :

```sql
-- Groupe "Sauces" pour l'article 42, de 0 à 2 choix
INSERT INTO modifier_groups (id, min_select, max_select, menu_item_id)
VALUES (10, 0, 2, 42);
-- "Crème" gratuite, "Caramel" avec supplément de 50 centimes
INSERT INTO modifiers (id, group_id, price_cents, sort_order)
VALUES (100, 10, 0, 0), (101, 10, 50, 1);

INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('modifier_group', 10, 'name', 'fr', 'Sauces'),
    ('modifier', 100, 'name', 'fr', 'Crème'),
    ('modifier', 101, 'name', 'fr', 'Caramel');
```

6. Gestion des utilisateurs

6.1 Créer un utilisateur

Premiers comptes, dans la graine du lot (au moins le Gérant) :

```sql
INSERT INTO users (insz, name, role, active)
VALUES ('85073003328', 'Marie Dupont', 'cashier', 1);
```

Rôles autorisés : `admin`, `cashier`, `waiter`. Voir `PROFILS_UTILISATEURS_FR.md` pour le mapping SCE 2.0. Les comptes suivants sont ajoutés par un Gérant dans "Config", panneau "Employés" ("Ajouter l'employé" ; un nouveau Gérant se confirme par le PIN du Gérant connecté, `MANUEL_UTILISATEUR_FR.md` §12.3). Ne pas créer l'INSZ `00000000029` (identité système).

6.2 Définir un code PIN

La graine du jeu de test du SPF porte des PIN hachés avec bcrypt (coût 12), calculés sur le poste du producteur :

```
htpasswd -bnBC 12 "" "1234" | tr -d ':\n'
# Sortie : $2y$12$abcdef... , à placer dans users.pin_hash de la graine
```

Pour un établissement, **aucun PIN dans la graine** : le premier Gérant choisit le sien à sa première connexion (“Choisissez votre PIN”), les autres fixent le leur en touchant leur nom ou le reçoivent d’un Gérant (§6.4).

6.3 Désactiver un utilisateur

Dans “Config”, panneau “Employés”, bouton “Désactiver” puis confirmation (un Gérant ne se désactive pas lui-même). L’utilisateur ne peut plus se connecter. Les événements fiscaux qu’il a générés restent intacts (l’INSZ est conservé comme identifiant historique).

Attention : Ne JAMAIS faire `DELETE FROM users` pour un utilisateur ayant des événements fiscaux à son nom - la FK depuis `fiscal_events` empêcherait la suppression, et même si elle réussissait par cascade (non configurée ici), cela violerait l’obligation de conservation.

6.4 Réinitialiser un PIN oublié

Un Gérant ouvre “Config”, panneau “Employés”, bouton “Code PIN” de l’employé, choisit le nouveau PIN et le confirme par son propre PIN ; pour un Gérant, c’est un **autre** Gérant. Aucun accès SQL ne remet un PIN à vide. Si le seul Gérant a oublié son PIN, la caisse n’offre pas de remise à zéro sur place : le distributeur décide de la suite avec le SPF Finances, sans réinitialiser ni désinstaller le terminal (§10.2) ; procédure de reprise : À effectuer (EV-10 : parcours d’un Gérant unique sans PIN sur les profils du D3 Pro).

Le compteur de PIN erronés (5 essais, puis 30 s doublées jusqu’à 15 min) est tenu en mémoire : éteindre puis rallumer le terminal le remet à zéro.

7. Période de réservation

Une période de réservation (`booking_period`) court du premier événement fiscal qui suit un rapport Z jusqu’au Z suivant. Elle est enregistrée dans la table `booking_periods` (un identifiant, la date de réservation de son ouverture et, une fois close, le Z qui la ferme). Une seule période est ouverte à la fois ; une période close ne se rouvre pas et ne se supprime pas (déclencheurs SQL). Redémarrer la caisse ne coupe donc jamais une journée.

La date de réservation est le jour commercial belge à l’ouverture : un service qui dépasse minuit reste sur sa journée jusqu’à l’heure `day_end_hour` de `pos.json` (5 h par défaut). Une période restée ouverte au-delà est clôturée par un Z automatique. Aucune modification SQL n’est nécessaire ni permise.

8. Désactiver / activer le mode formation

Le mode formation est un drapeau de `pos.json`, qui donne la valeur de départ au démarrage (`"training_mode": false`, ou `true` pour un lot de formation). Il se fixe dans le lot (`MANUEL_INSTALLATION_FR.md` §4.2), pas sur la caisse.

Un Gérant peut aussi le basculer dans “Config” (panneau “Mode Formation”, le bouton passe de “Inactif” à “Actif”), sans redémarrage et seulement quand aucune commande n’est ouverte avec des lignes (`MANUEL_UTILISATEUR_FR.md` §13). Ce réglage reste en mémoire : un redémarrage du terminal revient à la valeur du lot.

Attention : Ne JAMAIS laisser `training_mode: true` à la livraison au client - toutes les ventes deviendraient des événements T (formation) et NE compteraient PAS fiscalement.

9. Modifier les informations du restaurant

`restaurant_name`, `restaurant_address`, `restaurant_vat` et `restaurant_est_no` sont des champs de `pos.json`, donc du lot signé : ils se fixent au banc. L’écran d’import n’est ouvert que lorsque le moteur est fermé pour “Appareil non préparé” ou “Configuration d’un autre appareil” (`MANUEL_INSTALLATION_FR.md` §3.4)

; ré-import d'un lot corrigé sur un terminal en service : À effectuer (EV-10 : parcours de ré-import d'un lot signé sur les profils du D3 Pro). Aucune modification par fichier ou par SQL n'existe.

Attention : Modifier la TVA ou le `pos_id` après mise en production implique une déclaration au SPF Finances et potentiellement une re-certification.

10. Sauvegarde et restauration

10.1 Sauvegardes

Pas de sauvegarde manuelle sur la caisse. Elle écrit elle-même une copie de `pos.db` dans le stockage privé de l'application après chaque rapport Z, et l'outil de mise à jour une avant chaque installation (`GARANTIE_MAJ_LANGUES_FR.md` §2.1). Elle **conserve les 7 plus récentes** ; le journal fiscal de `pos.db` n'est jamais supprimé (`SECURITE_BD_FR.md` §5 ; commit du moteur : `4ff25b110c975d497f548538a08d81387b504b25` (commit « feat(engine): keep only the 7 newest database backups » du 2026-10-10, contenu dans l'étiquette de gel android-v1.0.0-rc3)). Une copie fraîche sort par l'export du Gérant vers une clé USB ou une carte SD (`SECURITE_BD_FR.md` §8.2) ; la base n'est jamais sur la carte (`PLATEFORME_ANDROID_FR.md` §5.1).

10.2 Restauration : jamais par-dessus `pos.db`

Ne jamais restaurer une sauvegarde par-dessus `pos.db`. Les événements fiscaux enregistrés depuis la sauvegarde seraient perdus, ce qui est interdit ; et les fichiers `pos.db-wal` et `pos.db-shm` qui accompagnent la base ne correspondraient plus. Les sauvegardes servent à consulter une situation passée (`sqlite3 -readonly` sur l'ordinateur, à partir de l'export) et à archiver, pas à revenir en arrière. La caisse n'offre d'ailleurs aucune restauration.

En cas de panne ou de corruption avérée : ne rien supprimer ni remplacer, **ne pas réinitialiser le terminal ni désinstaller l'application** (la base disparaîtrait avec elle), exporter ce qui peut l'être et contacter le distributeur, qui décide de la suite avec le SPF Finances. Pas de retour à une version antérieure : une version défectueuse se corrige par une nouvelle version déclarée (`GARANTIE_MAJ_LANGUES_FR.md` §2.1).

11. Migrations de schéma

Le schéma SQLite évolue via le runner versionné (`crates/pos-db/src/migrate.rs`). Les migrations sont automatiques au démarrage : si une nouvelle version est nécessaire, elle est appliquée dans une transaction `BEGIN IMMEDIATE` et tracée dans la table `_migrations_applied` (lisible dans la copie de `pos.db` de l'export : `SELECT version, name, applied_at FROM _migrations_applied ORDER BY version;`). Les migrations 60 à 63 (réglages du propriétaire et alertes, allergènes, service en plusieurs temps, bons cadeaux) font partie du moteur certifié (*SCHEMA_BD_FR.md* §2.9). Toute nouvelle migration d'une version pili-pos-android suivra le même mécanisme - aucune intervention manuelle n'est requise du distributeur sur site.

12. Diagnostic et logs

12.1 Logs de la caisse

Pas de `journalctl` ni de terminal : les journaux du moteur (un fichier par jour) et le journal d'audit de la plateforme ne se lisent que dans l'export du Gérant vers le SPF, dossiers `logs/` et `audit/` (`SECURITE_BD_FR.md` §8.2). L'état de la caisse (FDM, imprimante, version, empreintes, espace libre) se lit à l'écran de connexion et dans "À propos" (§15).

12.2 Diagnostic FDM

Depuis le poste du technicien, sur le même LAN que le FDM, avec les fichiers du lot (module2 : HTTPS 443, certificat client mTLS, jeton Bearer ; `-k` car le certificat du FDM ne porte pas de nom d'hôte, la caisse le vérifie par l'autorité `fdm_ca_file`) :

```
curl -k -X POST https://192.168.1.20/graphql/ \
  --cert lot/fdm/client.crt --key lot/fdm/client.key \
  -H "Authorization: Bearer $(cat lot/secrets/fdm.token)" \
  -H "Content-Type: application/json" \
  -d '{"query": "query { status(language: FR) { device { fdmId fdmDateTime } } }"}'
```

Réponse attendue : JSON contenant l'identité du FDM. Si pas de réponse : problème réseau ou FDM hors-ligne. La caisse interroge d'abord `status` en minuscules, puis `Status` si le FDM refuse cette orthographe. Un FDM en HTTP (port 80) se teste avec une URL `http://` et sans `--cert` ni `--key`. "Not a valid Token" indique un jeton absent ou erroné. Messages de la caisse : `MANUEL_UTILISATEUR_FR.md` §14.

12.3 Diagnostic imprimante

```
nc -zv 192.168.1.30 9100
echo -e "Test impression\n\n\n\x1bm" | nc -w 1 192.168.1.30 9100
```

12 bis. Fonctions du propriétaire : réglages et diagnostic

Quatre fonctions non fiscales ou annexes (migrations 60 à 63, *SCHEMA_BD_FR.md* §2.9) se règlent à l'écran, jamais en SQL. Le tableau dit où lire leur état.

Fonction	Réglage (écran « Config », Gérant)	État lisible en SQL	Annexe
Alertes sur téléphone	panneau « Alertes sur téléphone » ; jeton dans le fichier `alerts.telegram_bot_token_file` de `pos.json`	`owner_settings` (clé `alerts`), `owner_alert_log`	*ALERTES_PROPRIETAIRE_FR.md*
Service en plusieurs temps	panneau « Service en plusieurs temps »	`owner_settings` (clé `courses`), `order_line_courses`, `order_course_releases`, `kitchen_slip_entries`	*SERVICES_CUISINE_FR.md*
Bons cadeaux	panneau « Bons cadeaux »	`owner_settings` (clé `gift_vouchers`), `gift_vouchers`, `gift_voucher_movements`, `gift_voucher_prints`	*BONS_CADEAUX_FR.md*
Allergènes	éditeur de carte, ou catalogue HubRise	`menu_item_allergens`, `modifier_allergens`, `allergen_declarations`, `allergen_change_log`	*ALLERGENES_FR.md*

Une clé absente de `owner_settings` veut dire « fonction éteinte ». Les journaux (`owner_settings_log`, `owner_alert_log`, `allergen_change_log`) et les trois tables de bons cadeaux sont en ajout seul : leurs déclencheurs refusent toute modification et toute suppression. Lectures utiles (lecture seule, sur une copie de la base) :

```
-- Derniers messages d'alerte et leur résultat (le jeton n'y est jamais)
SELECT created_at, kind, transition, outcome, http_status
FROM owner_alert_log ORDER BY id DESC LIMIT 20;

-- Solde et état d'un bon : somme de ses mouvements
```

```sql
SELECT v.id, v.expires_on, SUM(m.amount_cents) AS solde_centimes
FROM gift_vouchers v JOIN gift_voucher_movements m ON m.voucher_id = v.id
GROUP BY v.id;
```

Un message test (`Envoyer un message test`) vérifie la conversation sans attendre une panne. Si les alertes ne partent pas, le journal dit pourquoi (`failed` avec le code HTTP, ou `suppressed` après 20 messages dans l'heure).

Hors de la caisse. L'export pour le comptable est un outil séparé, `pili-accounting` (`crates/pos-accounting`), qui ne fait pas partie du binaire certifié ni du paquet livré avec la caisse : il lit seulement une copie de l'export ou d'une sauvegarde, jamais la base en service, et ne signe rien. Il ne s'exécute pas sur la caisse.

13. Procédure de release par le distributeur

À partir du dépôt git (détail et garanties : `GARANTIE_MAJ_LANGUES_FR.md` §1 et §2) :

```
# 1. Tout commité, tests verts (moteur, android/rust, application Kotlin)
cargo test --workspace && cargo clippy --workspace -- -D warnings
# 2. Tag git signé GPG
git tag -s android-vX.Y.Z -m "pili-pos-android vX.Y.Z – <résumé>"
# 3. Deux builds du même commit dans le conteneur épinglé, hors réseau :
#    mêmes APK non signés, puis contrôle de forme (check_apk.sh)
scripts/android/check_repro.sh
# 4. Signature (clé RSA de release), SHA-1 de l'APK signé, déclaration au
#    SPF Finances (délai de 15 jours)
# 5. Après son accord seulement : publication de l'artefact déclaré
scripts/release/declare_android.sh --apk <APK non signé> --version X.Y.Z \
  --sha1 <SHA-1 accepté> --keystore <clé de release> --alias <alias>
```

La clé de signature de release et l'étape 5 appartiennent au producteur, jamais à une exécution automatique.

14. Anti-pattern à éviter

NON NE PAS FAIRE	✓ FAIRE À LA PLACE
`DELETE FROM fiscal_events`	Aucune suppression - exigence légale
`DELETE FROM users WHERE insz = ...`	Bouton "Désactiver" de "Config"
`UPDATE fiscal_events SET ...`	Aucune modification - émettre une correction via événement séparé
`DELETE FROM orders WHERE status = 'closed'`	Conservation indéfinie
Bypass de FDM en production	Bloquer la vente - exigence cert
`training_mode: true` en production	Toujours `false` dans le lot livré
Suppression d'une catégorie ayant des items	`active = 0`
Modifier un menu géré par HubRise ailleurs que dans HubRise	Modifier le produit dans HubRise (la synchronisation écrase le reste, §4.5)
Modifier en SQL `owner_settings`, `gift_vouchers` ou `allergen_declarations`	Régler à l'écran (§12 bis) : les journaux inaltérables gardent la trace et les déclencheurs refusent la modification des bons
Restaurer une sauvegarde par-dessus `pos.db`	Ne jamais restaurer ; contacter le distributeur (§10.2)
Réinitialiser aux paramètres d'usine ou désinstaller un terminal en service	La base fiscale disparaît avec l'application : exporter et contacter le distributeur (§10.2)
Se connecter à un compte Google, installer un APK par adb	Aucun compte ; seule la mise à jour déclarée installe l'APK (`MANUEL_INSTALLATION_FR.md` §3.5 et §6)
Changement de `pos_id` ou `restaurant_vat` sans déclaration SPF	Procédure post-cert obligatoire

15. Réglages à l'écran de la caisse

Ce que la caisse permet de changer en exploitation, avec les libellés de l'écran Paramètres (français par défaut). Les règles (qui peut créer un compte, vérification des PIN, refus de couper HubRise avec des commandes en ligne non terminées) sont celles du moteur (`docs/android/parity/settings.md`).

Panneau ou ligne	Qui	Ce qu'on y règle
"Mode Formation"	Gérant	bascule du mode, "Inactif" ou "Actif" (§8)
"Langue"	tous	Français, English, 中文 (§2)
"Affichage des produits"	tous	"Nom" ou "Code" (le code du menu remplace le nom à l'écran, les tickets ne changent pas)
"Employés"	Gérant	liste, "Ajouter l'employé", "Code PIN", "Désactiver" (§6) ; un employé fixe son propre PIN en touchant son nom
"Informations"	tous	lecture seule : POS ID, Terminal, Restaurant, FDM, Imprimante
"Tickets par email"	tous	état des envois (`MANUEL_UTILISATEUR_FR.md` §12.5)
"Intégrations"	interrupteur : Gérant	"Activer" ou "Désactiver" HubRise, "Historique" (§4.5)
"HubRise"	Gérant, HubRise activé	connexion, catalogue, "Reprendre le menu", "Canaux"
"Autres réglages"	selon la ligne	"À propos" (tous), "Export pour le SPF", "Mise à jour" (Gérant), "Export nocturne"

Tout le reste (identité fiscale, FDM, imprimantes, `language`, `day_end_hour`, `table_count`, blocs facultatifs) est dans `pos.json` et ne se change pas sur la caisse (§9). Pas d'écran de réseau, pas d'assistance à distance (`PLATEFORME_ANDROID_FR.md` §7).